

E-COMMERCE BUSINESS PERFORMANCE ANALYSIS

SQL Portfolio Project

PostgreSQL • pgAdmin • Olist Brazilian E-Commerce Dataset

Kristoffer Fields

Project Brief

A practical SQL analysis of marketplace revenue, customer behavior, product performance, and delivery experience using real anonymized transaction data.

September 2026

Project Overview

I built this project to practice the kind of SQL work that comes up in a business analyst or data analyst role. Instead of working from one flat spreadsheet, the dataset is split across customers, orders, products, sellers, payments, reviews, and order items. That makes it useful for practicing joins and building metrics from relational data.

The goal is straightforward: use SQL to turn the raw tables into a small set of business metrics, then explain what those numbers say about marketplace performance. The analysis starts with revenue and order trends, then moves into customer behavior, product performance, seller activity, and delivery experience.

Dataset	Period	Scale	Database
Olist Brazilian E-Commerce	2016-2018	~100K orders	PostgreSQL

Dataset

The project uses the Brazilian E-Commerce Public Dataset by Olist. The release contains anonymized commercial order data from a Brazilian marketplace. It covers roughly 100,000 orders placed between 2016 and 2018 and includes enough detail to look at both sales performance and the customer experience.

Core tables used in the project:

- customers - customer IDs and location information
- orders - order status and purchase / delivery timestamps
- order_items - product, seller, item price, and freight value
- products - product category and physical product attributes
- sellers - seller IDs and location information
- payments - payment method, installments, and payment value
- reviews - review scores and customer comments

Two additional files - geolocation data and a Portuguese-to-English category translation table - can be added later when the analysis reaches geographic detail and final presentation work.

Analytics Table

To make the first round of analysis easier, I created an order-level table called `order_summary`. It joins the `orders`, `customers`, and `order_items` tables and rolls item-level records up so that each row represents one order.

The table currently includes:

- order and customer identifiers
- customer city and state
- order status and purchase timestamp
- product revenue
- freight revenue
- total order value
- number of items in the order

This gives me a clean starting point for KPIs such as total revenue, monthly revenue, average order value, order volume, and state-level performance without changing the original raw tables.

Business Questions

The project is organized around questions that could reasonably come from a manager rather than around a checklist of SQL commands.

Revenue & Growth

- How much product revenue was generated during the period?
- How did revenue and order volume change month to month?
- What is the average order value, and does it change over time?
- Which states and product categories contribute the most revenue?

Customer Behavior

- How many unique customers purchased through the marketplace?
- What share of customers made more than one purchase?
- Which locations have the highest average order values?
- How does purchase frequency vary across customers?

Product & Seller Performance

- Which categories generate the most revenue and order volume?
- Which sellers account for the most sales?
- Which categories sell frequently but have relatively low order values?
- How concentrated is revenue across categories and sellers?

Operations & Customer Experience

- How long does delivery take?
- How often do orders arrive after the estimated delivery date?
- Do late deliveries receive lower review scores?
- Which payment methods and installment patterns are most common?

SQL Skills Used

The project starts with basic aggregation and becomes more advanced as the questions require it. The point is not to use complicated SQL just to make the project look technical; each technique should solve a specific analysis problem.

Area	SQL used
Core querying	SELECT, WHERE, ORDER BY, DISTINCT
Business metrics	COUNT, SUM, AVG, GROUP BY, date functions
Relational analysis	JOINS across customers, orders, products, sellers, payments, and reviews
Logic and transformations	CASE WHEN, subqueries, common table expressions (CTEs)
Trend and ranking analysis	window functions, RANK, ROW_NUMBER, LAG

Current Progress

- PostgreSQL database created and Olist CSV files imported into relational tables.
- order_summary analytical table created from orders, customers, and order_items.
- Total product revenue calculated: \$13,591,643.70.
- Core analysis completed across revenue, customers, categories, sellers, delivery, reviews, payment methods, and installments.
- Project connection: pgAdmin server "Local PostgreSQL"; host localhost; port 5432; user postgres; database olist_ecommerce.

Analysis & Results

The sections below are the working structure for the rest of the project. As each analysis is completed, the final SQL, result, chart, and short interpretation can be dropped into the matching section.

Revenue Performance

This section will answer the following business questions:

- How much revenue did the marketplace generate?
- How did revenue and order volume change over time?
- What was the average order value?

1. Total Revenue

Business question: How much product revenue was generated during the analysis period?

SQL approach: Used the order_summary analytical table and SUM(product_revenue) to calculate total product revenue across the full dataset.

```
SELECT
  ROUND(SUM(product_revenue)::numeric, 2) AS total_product_revenue
FROM order_summary;
```

Metric / Output	Result	Interpretation
Total product revenue	\$13,591,643.70	Order-item prices sum to about \$13.59 million in product revenue across the historical marketplace sample.

KPI: \$13.59M in product revenue across the historical dataset.

Finding: The marketplace generated \$13.59 million in product revenue during the dataset period.

2. Monthly Revenue Trend

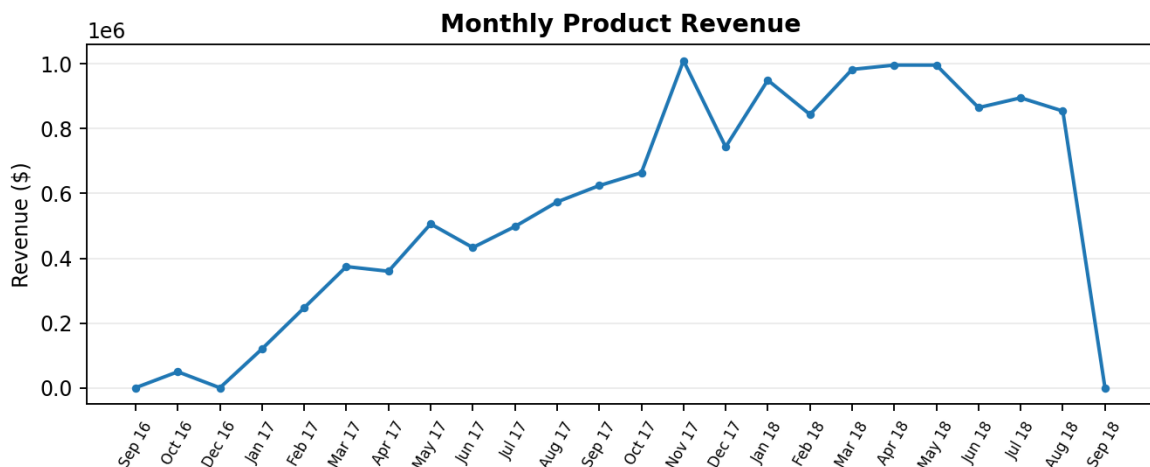
Business question: How did product revenue change from month to month?

SQL approach: Aggregated order_summary by purchase month, calculated monthly product revenue and distinct order volume, and used LAG() to calculate month-over-month revenue growth.

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_order_purchase_timestamp)::date AS month,
    ROUND(SUM(product_revenue)::numeric, 2) AS monthly_revenue,
    COUNT(DISTINCT order_id) AS monthly_orders
  FROM order_summary
  GROUP BY 1
),
with_growth AS (
  SELECT
    month,
    monthly_revenue,
    monthly_orders,
    LAG(monthly_revenue) OVER (ORDER BY month) AS prior_month_revenue
  FROM monthly
)
SELECT
  month,
  monthly_revenue,
  monthly_orders,
  ROUND(
    100.0 * (monthly_revenue - prior_month_revenue)
    / NULLIF(prior_month_revenue, 0), 2
  ) AS mom_revenue_growth_pct
FROM with_growth
```

ORDER BY month;

Metric / Output	Result	Interpretation
Monthly revenue	\$1,010,271.37 peak monthly revenue (Nov. 2017)	Monthly product revenue rose substantially as marketplace activity scaled, reaching its highest observed level in November 2017. Much of 2018 remained near the \$0.85-\$1.00 million monthly range.
Monthly order volume	7,451 orders peak monthly volume (Nov. 2017)	Order volume generally increased alongside revenue, with November 2017 recording the highest monthly order count in the analysis.
Month-over-month growth	+52.10% in Nov. 2017; -26.36% in Dec. 2017	Month-over-month growth was volatile. Very early and final dataset months contain extremely low order counts, so their percentage changes are not representative of normal marketplace performance.



Finding: Revenue and order volume scaled rapidly through 2017, peaking in November 2017 at \$1.01 million in product revenue and 7,451 orders. Monthly revenue then remained relatively high through most of 2018, while boundary months with very few orders produced misleadingly extreme growth rates.

3. Average Order Value

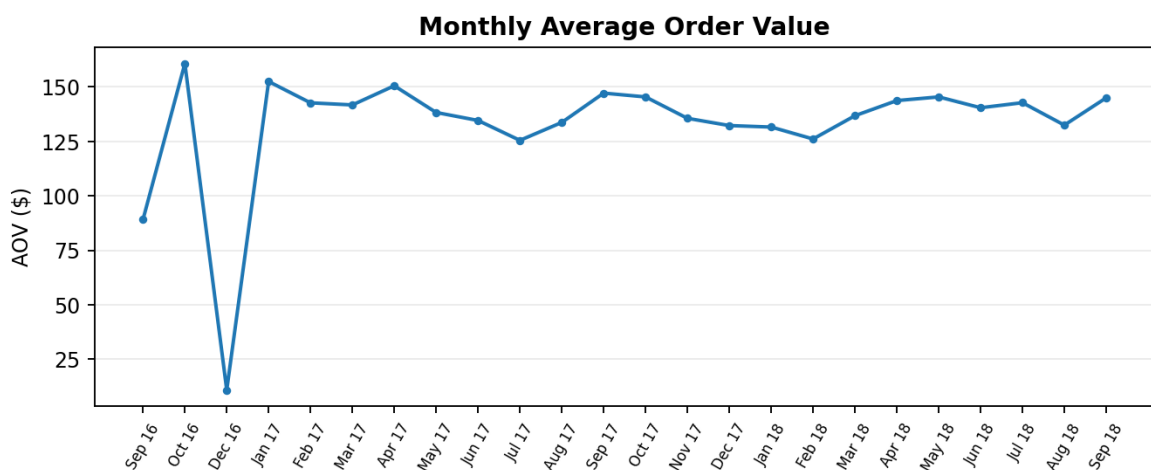
Business question: What was the average product value per order, and how did it change over time?

SQL approach: Calculated product revenue per order from order_summary, then averaged those order totals overall and by purchase month.

```
-- Overall average order value
SELECT
  ROUND(AVG(product_revenue)::numeric, 2) AS overall_avg_order_value
FROM order_summary;

-- Monthly average order value
SELECT
  DATE_TRUNC('month', order_order_purchase_timestamp)::date AS month,
  ROUND(AVG(product_revenue)::numeric, 2) AS monthly_avg_order_value
FROM order_summary
GROUP BY 1
ORDER BY 1;
```

Metric / Output	Result	Interpretation
Overall average order value	\$137.75	Across 98,666 orders, average product revenue per order was \$137.75.
Monthly average order value	Generally ~\$125-\$150 in established 2017-2018 months	Monthly AOV was much steadier than monthly revenue, suggesting growth was driven mainly by more orders rather than rapidly rising spend per order.



Finding: Average product revenue per order was \$137.75. After the very low-volume launch months, monthly AOV was comparatively stable, generally around \$125-\$150, indicating that revenue growth was driven more by rising order volume than by higher spending per order.

Product & Category Performance

This section will answer the following business questions:

- Which product categories generate the most revenue?
- Which categories receive the most orders?
- Which categories combine high volume with high order value?

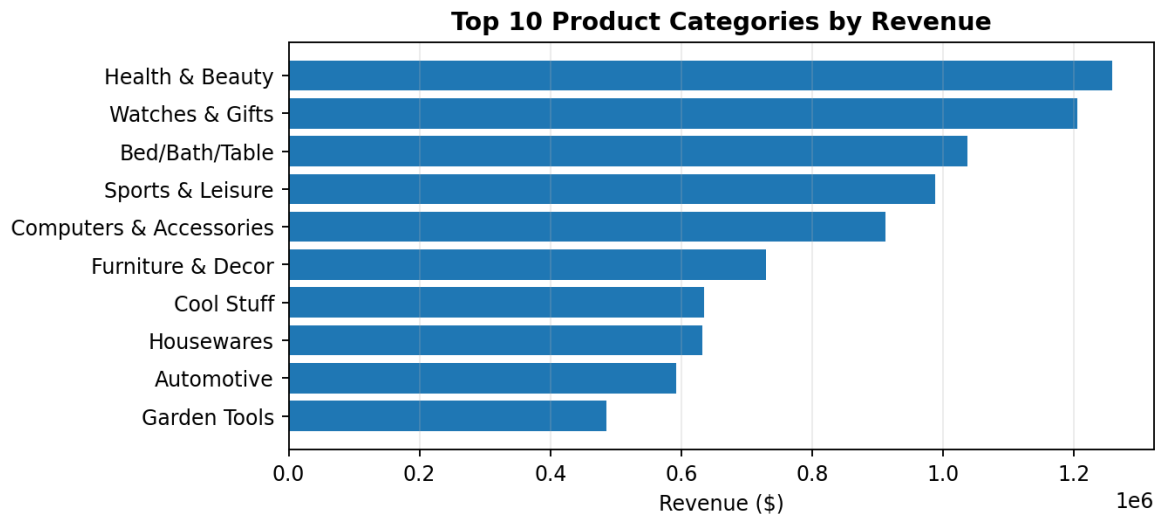
4. Category Revenue

Business question: Which product categories contribute the most revenue?

SQL approach: Joined order_items to products, grouped by product category, summed item price as product revenue, and calculated each category's share of total revenue.

```
WITH category_sales AS (  
  SELECT  
    COALESCE(p.product_category_name, 'unknown') AS category,  
    SUM(oi.price) AS category_revenue  
  FROM order_items oi  
  LEFT JOIN products p ON oi.product_id = p.product_id  
  GROUP BY 1  
,  
total AS (  
  SELECT SUM(category_revenue) AS total_revenue  
  FROM category_sales  
)  
SELECT  
  category,  
  ROUND(category_revenue::numeric, 2) AS category_revenue,  
  ROUND(100.0 * category_revenue / total_revenue, 2) AS revenue_share_pct  
FROM category_sales  
CROSS JOIN total  
ORDER BY category_revenue DESC  
LIMIT 10;
```

Metric / Output	Result	Interpretation
Top categories by revenue	Health & Beauty: \$1,258,681.34	Health & Beauty led category revenue, followed by Watches & Gifts (\$1.21M) and Bed, Bath & Table (\$1.04M).
Share of revenue from top categories	Top 5 categories: ~39.7% of total revenue	A relatively small group of categories accounted for a substantial share of marketplace product revenue.



Finding: Health & Beauty was the highest-revenue category at \$1.26 million, followed by Watches & Gifts (\$1.21 million) and Bed, Bath & Table (\$1.04 million). The top five categories generated about 39.7% of total product revenue.

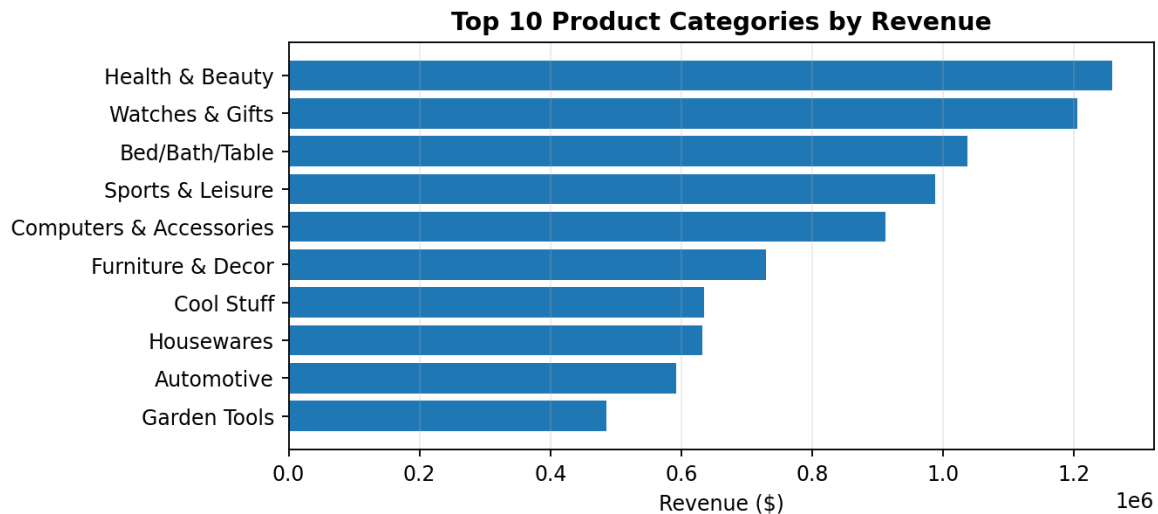
5. Category Order Volume

Business question: Which categories are purchased most frequently?

SQL approach: Joined order_items to products and compared item count with distinct order count by product category.

```
SELECT
  COALESCE(p.product_category_name, 'unknown') AS category,
  COUNT(*) AS item_count,
  COUNT(DISTINCT oi.order_id) AS order_count
FROM order_items oi
LEFT JOIN products p ON oi.product_id = p.product_id
GROUP BY 1
ORDER BY item_count DESC
LIMIT 10;
```

Metric / Output	Result	Interpretation
Top categories by item count	Bed, Bath & Table: 11,115 items	Bed, Bath & Table had the highest item count, ahead of Health & Beauty (9,670) and Sports & Leisure (8,641).
Top categories by orders	Bed, Bath & Table: 9,417 orders	The same category also appeared in the most distinct orders, while Health & Beauty ranked second with 8,836 orders.



Finding: Bed, Bath & Table had the highest item count (11,115) and the most distinct orders (9,417). Health & Beauty ranked second in volume but first in revenue, showing that sales volume and revenue leadership are related but not identical.

Customer & Geographic Performance

This section will answer the following business questions:

- How many unique customers purchased through the marketplace?
- What share of customers purchased more than once?
- Which states account for the most revenue and customers?

6. Customer Base

Business question: How large is the customer base and how often do customers return?

SQL approach: Joined customers to orders using customer_id, then used customer_unique_id to identify unique people and count repeat purchasers across multiple orders.

```
WITH customer_orders AS (
  SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id) AS order_count
  FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  GROUP BY c.customer_unique_id
)
SELECT
  COUNT(*) AS unique_customers,
  COUNT(*) FILTER (WHERE order_count > 1) AS repeat_customers,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE order_count > 1) / COUNT(*),
    2
  ) AS repeat_purchase_rate_pct
```

```
FROM customer_orders;
```

Metric / Output	Result	Interpretation
Unique customers	96,096	The dataset contains 96,096 unique customer identities based on customer_unique_id.
Repeat customers	2,997	Only 2,997 customers placed more than one order.
Repeat purchase rate	3.12%	Repeat purchasing was uncommon; most customers made only one observed purchase.

Customer retention KPI: 3.12% repeat purchase rate.

Finding: The marketplace served 96,096 unique customers, but only 2,997 made more than one purchase, producing a 3.12% repeat purchase rate. Most observed customers therefore purchased only once.

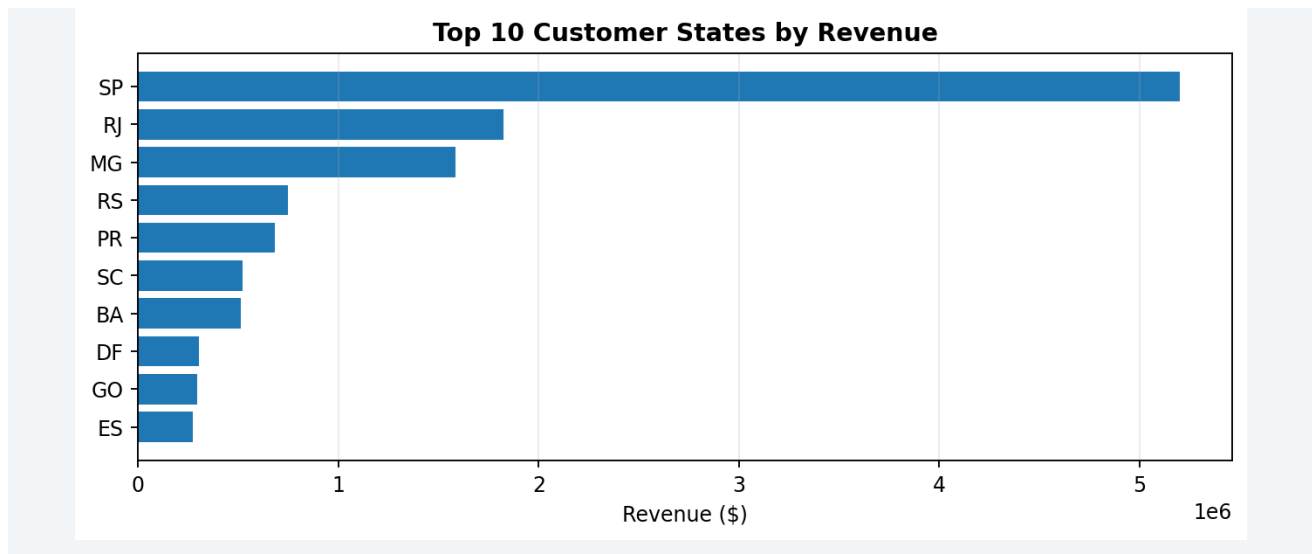
7. State-Level Performance

Business question: Which customer states generate the most revenue and order volume?

SQL approach: Used order_summary to aggregate product revenue, distinct orders, and average order value by customer state.

```
SELECT
  customer_state,
  ROUND(SUM(product_revenue)::numeric, 2) AS product_revenue,
  COUNT(DISTINCT order_id) AS orders,
  ROUND(AVG(product_revenue)::numeric, 2) AS avg_order_value
FROM order_summary
GROUP BY customer_state
ORDER BY product_revenue DESC;
```

Metric / Output	Result	Interpretation
Revenue by state	SP \$5.20M; RJ \$1.82M; MG \$1.59M	São Paulo generated about 38.3% of all product revenue; the top three states together generated about 63.4%.
Orders by state	SP 41,375; RJ 12,762; MG 11,544	Order volume was also heavily concentrated in the same major customer markets.
Average order value by state	PB \$216.67; AP \$198.15; AC \$197.32	Some smaller states had higher AOVs, but their low order counts mean they contributed far less total revenue than SP, RJ, and MG.



Finding: Revenue was geographically concentrated. São Paulo generated \$5.20 million (about 38.3% of total product revenue), while São Paulo, Rio de Janeiro, and Minas Gerais together generated about 63.4%.

Seller Performance

This section will answer the following business questions:

- Which sellers generate the most revenue?
- How concentrated are sales among the top sellers?

8. Seller Revenue

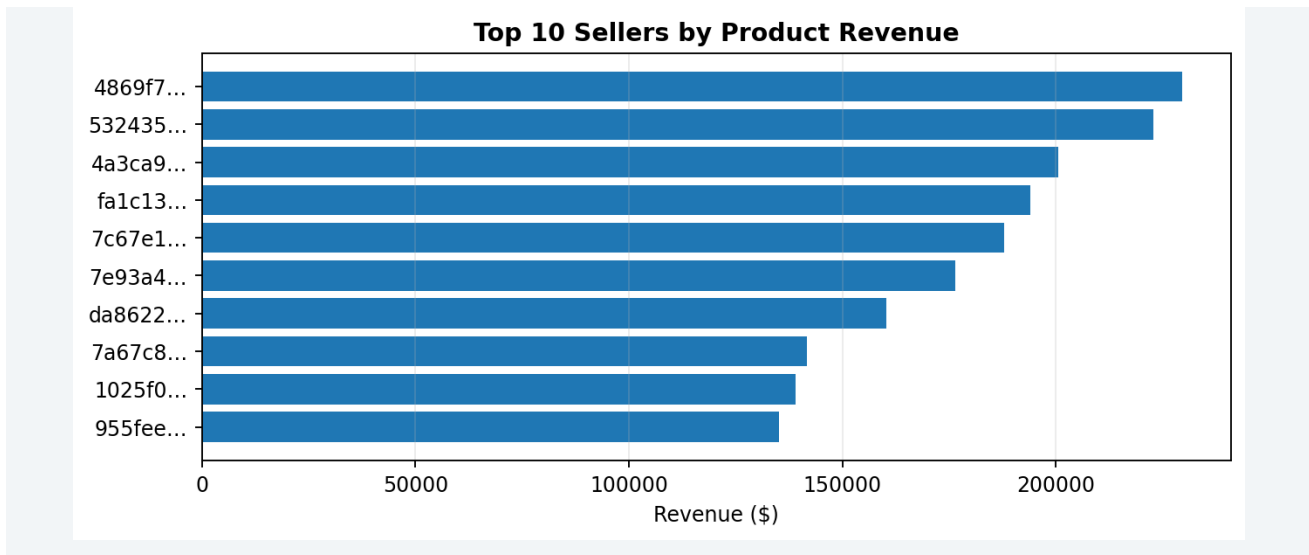
Business question: Which sellers account for the most marketplace sales?

SQL approach: Aggregated order_items by seller_id to calculate seller product revenue and distinct order count, then ranked sellers by performance.

```
SELECT
  seller_id,
  ROUND(SUM(price)::numeric, 2) AS seller_revenue,
  COUNT(DISTINCT order_id) AS order_count,
  COUNT(*) AS items_sold
FROM order_items
GROUP BY seller_id
ORDER BY seller_revenue DESC
LIMIT 10;
```

Metric / Output	Result	Interpretation
Top sellers by revenue	Top seller: \$229,472.63	The leading seller generated \$229.5K, narrowly ahead of the second-ranked seller at \$222.8K.

Top sellers by order count	Highest order count among top 10: 1,806	High seller revenue came from different strategies: some sellers relied on large order volume, while others sold higher-priced items.
-----------------------------------	--	---



Finding: Seller performance reflected both volume and price. The highest-revenue seller generated \$229,472.63 from 1,132 orders, while the second-ranked seller generated \$222,776.05 from only 358 orders because its average item price was much higher.

9. Revenue Concentration

Business question: How much of marketplace revenue comes from the highest-performing sellers?

SQL approach: Aggregated revenue by seller, ranked sellers, and compared cumulative revenue from the top 10 and top 25 sellers with total marketplace product revenue.

```

WITH seller_sales AS (
  SELECT
    seller_id,
    SUM(price) AS revenue
  FROM order_items
  GROUP BY seller_id
),
ranked AS (
  SELECT
    seller_id,
    revenue,
    ROW_NUMBER() OVER (ORDER BY revenue DESC) AS seller_rank
  FROM seller_sales
),
totals AS (
  SELECT SUM(revenue) AS total_revenue FROM seller_sales
)
SELECT
  ROUND(100.0 * SUM(revenue) FILTER (WHERE seller_rank <= 10)
    / MAX(total_revenue), 2) AS top_10_share_pct,
  ROUND(100.0 * SUM(revenue) FILTER (WHERE seller_rank <= 25)

```

```

/ MAX(total_revenue), 2) AS top_25_share_pct
FROM ranked
CROSS JOIN totals;

```

Metric / Output	Result	Interpretation
Top 10 seller revenue share	13.15%	The 10 highest-revenue sellers generated 13.15% of total product revenue.
Top 25 seller revenue share	23.57%	The top 25 sellers generated 23.57%, indicating moderate seller concentration rather than extreme dependence on a handful of sellers.

Seller concentration: top 10 = 13.15% of revenue; top 25 = 23.57%.

Finding: The top 10 sellers generated 13.15% of marketplace product revenue, and the top 25 generated 23.57%. Revenue was concentrated among leading sellers, but not dominated by a very small handful of sellers.

Delivery & Customer Experience

This section will answer the following business questions:

- How long do deliveries take?
- How often are orders delivered late?
- Is delivery performance associated with review scores?

10. Delivery Performance

Business question: How quickly are completed orders delivered and how often do they miss the estimated date?

SQL approach: Used delivered orders with non-null delivery timestamps to calculate average delivery days and the percentage delivered after the estimated delivery date.

```

SELECT
  ROUND(AVG(EXTRACT(EPOCH FROM
    (order_delivered_customer_date - order_purchase_timestamp)
  ) / 86400.0)::numeric, 2) AS avg_delivery_days,
  ROUND(
    100.0 * COUNT(*) FILTER (
      WHERE order_delivered_customer_date > order_estimated_delivery_date
    ) / COUNT(*),
    2
  ) AS late_delivery_rate_pct
FROM orders
WHERE order_delivered_customer_date IS NOT NULL
AND order_purchase_timestamp IS NOT NULL

```

AND order_estimated_delivery_date IS NOT NULL;

Metric / Output	Result	Interpretation
Average delivery time	12.56 days	Completed orders took an average of 12.56 days from purchase to customer delivery.
Late delivery rate	8.11%	Fewer than one in ten delivered orders arrived after the estimated delivery date.

Delivery KPI: 12.56 average days; 8.11% late.

Finding: Delivered orders reached customers in an average of 12.56 days, and 8.11% arrived after the estimated delivery date. Delivery performance was generally reliable, although late orders represented a meaningful service-risk segment.

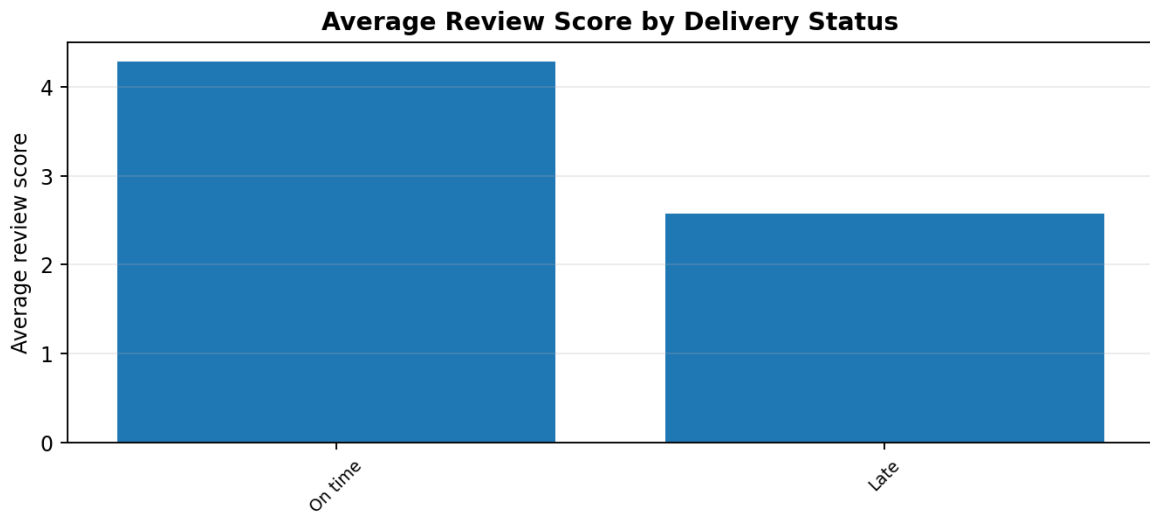
11. Reviews & Delivery

Business question: Do late deliveries tend to receive lower customer review scores?

SQL approach: Joined orders to reviews by order_id, classified delivered orders as on-time or late, and compared average review scores between the two groups.

```
SELECT
CASE
  WHEN o.order_delivered_customer_date <= o.order_estimated_delivery_date
    THEN 'On time'
  ELSE 'Late'
END AS delivery_status,
COUNT(*) AS review_count,
ROUND(AVG(r.review_score)::numeric, 2) AS avg_review_score
FROM orders o
JOIN reviews r ON o.order_id = r.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
  AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY 1
ORDER BY 1;
```

Metric / Output	Result	Interpretation
Average review score - on time	4.29 / 5	On-time deliveries received strong average customer ratings across 88,658 reviewed orders.
Average review score - late	2.57 / 5	Late deliveries averaged 1.72 review points lower across 7,701 reviewed orders.



Finding: On-time deliveries averaged a 4.29 review score versus 2.57 for late deliveries, a 1.72-point gap. Late delivery was strongly associated with lower customer satisfaction in this historical sample.

Payment Behavior

This section will answer the following business questions:

- Which payment methods are used most often?
- How common are installment payments?

12. Payment Methods

Business question: How do customers pay for marketplace orders?

SQL approach: Aggregated the payments table by payment_type to calculate usage share, total payment value, and average payment value by method.

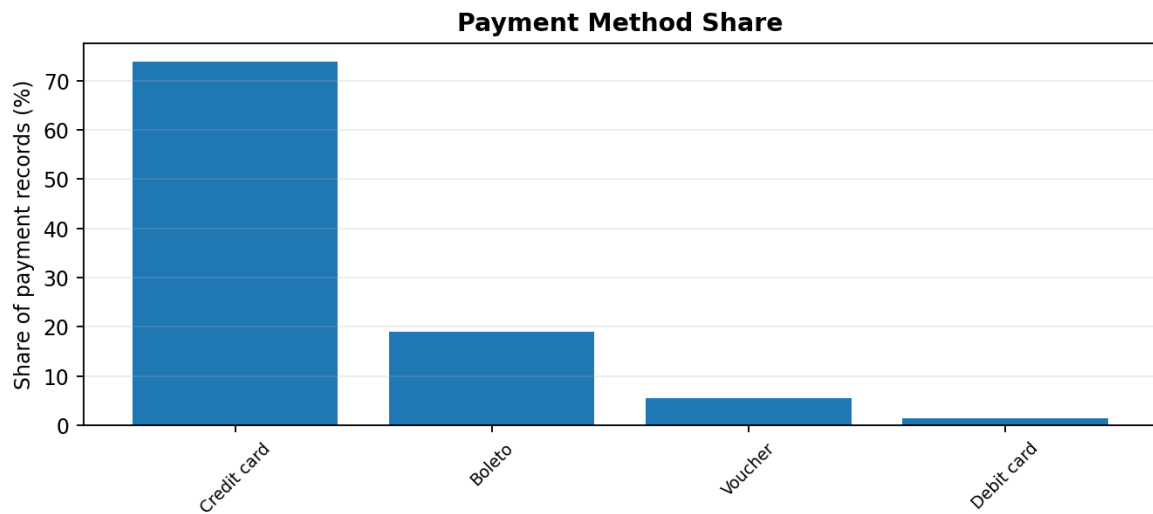
```
WITH payment_summary AS (
  SELECT
    payment_type,
    COUNT(*) AS payment_records,
    SUM(payment_value) AS total_payment_value,
    AVG(payment_value) AS avg_payment_value
  FROM payments
  GROUP BY payment_type
),
totals AS (
  SELECT SUM(payment_records) AS total_records FROM payment_summary
)
SELECT
  payment_type,
  payment_records,
  ROUND(100.0 * payment_records / total_records, 2) AS payment_method_share_pct,
  ROUND(total_payment_value::numeric, 2) AS total_payment_value,
```

```

ROUND(avg_payment_value::numeric, 2) AS avg_payment_value
FROM payment_summary
CROSS JOIN totals
ORDER BY payment_records DESC;

```

Metric / Output	Result	Interpretation
Payment method share	Credit card 73.92%; boleto 19.04%	Credit cards dominated payment usage; vouchers (5.56%) and debit cards (1.47%) were much less common.
Average payment value	Credit card \$163.32; boleto \$145.03	Credit cards also had the highest average payment value among the major payment methods.



Finding: Credit cards dominated marketplace payments, representing 73.92% of payment records and about \$12.54 million in total payment value. Boleto was a distant second at 19.04%.

13. Installments

Business question: How frequently are installments used, and for what kinds of order values?

SQL approach: Used payment_installments to calculate average installments, the share of payment records using more than one installment, and average payment value by installment count.

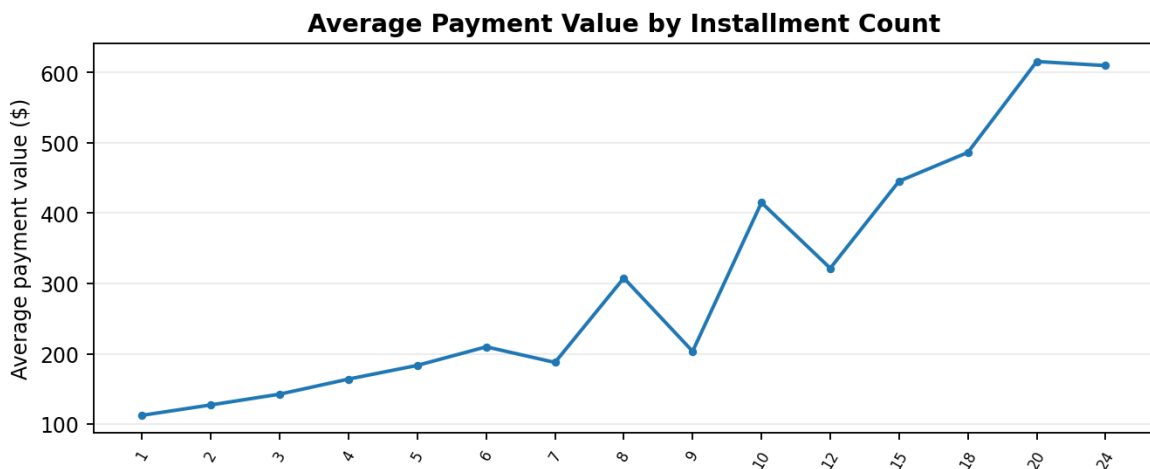
```

-- Overall installment metrics
SELECT
  ROUND(AVG(payment_installments)::numeric, 2) AS avg_installments,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE payment_installments > 1) / COUNT(*),
    2
  ) AS share_using_installments_pct
FROM payments
WHERE payment_installments IS NOT NULL;

```

```
-- Average payment value by installment count
SELECT
  payment_installments,
  COUNT(*) AS payment_records,
  ROUND(AVG(payment_value)::numeric, 2) AS avg_payment_value
FROM payments
WHERE payment_installments IS NOT NULL
GROUP BY payment_installments
ORDER BY payment_installments;
```

Metric / Output	Result	Interpretation
Average installments	2.85	The average payment record used 2.85 installments.
Share using installments	49.42%	Nearly half of payment records used more than one installment.
Average value by installment count	1x \$112.42; 5x \$183.47; 8x \$307.74; 10x \$415.09	Average payment value generally increased with installment count, consistent with customers spreading higher-value purchases across more payments.



Finding: Installment purchasing was common: the average payment used 2.85 installments and 49.42% of payment records used more than one installment. Higher installment counts were generally associated with higher average payment values.

Final Findings & Business Takeaways

Finding 1: Marketplace growth was driven primarily by scale. Product revenue totaled \$13.59 million, monthly revenue peaked at \$1.01 million in November 2017, and average order value remained relatively stable around the mid-\$100s during established months.

Finding 2: Revenue was concentrated geographically and by category. São Paulo generated about 38.3% of product revenue, while Health & Beauty, Watches & Gifts, Bed/Bath/Table, Sports & Leisure, and Computers & Accessories together generated about 39.7%.

Finding 3: Customer experience and retention present the clearest improvement opportunities. Only 3.12% of customers purchased more than once, and late deliveries averaged a 2.57 review score versus 4.29 for on-time deliveries.

Recommendation / next step: Prioritize retention analysis and delivery reliability. A useful next phase would segment repeat-purchase behavior by category and state, then identify sellers or routes with elevated late-delivery rates to target operational improvements.

Data Limitations

- The dataset covers 2016-2018, so the findings describe this historical marketplace sample rather than current Olist performance.
- Some orders contain missing lifecycle timestamps, and delivery analyses exclude records without required purchase, delivery, or estimated-delivery dates. Revenue analyses use the imported order-item records unless a section states otherwise.
- Product revenue is calculated from `order_items.price`. It is a gross sales measure and should not be interpreted as Olist company revenue, margin, or profit.

SQL Appendix

The final version can include the clean SQL used for the main analyses. Longer exploratory queries can stay in the GitHub repository rather than making the report unnecessarily long.

Database setup / table creation

```
-- Connection used for this project
-- pgAdmin server label: Local PostgreSQL
-- Host: localhost | Port: 5432 | User: postgres
-- Database: olist_ecommerce

-- Verify the active connection
SELECT current_database(), current_user, version();

-- Verify core tables
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

Order summary transformation

```
DROP TABLE IF EXISTS order_summary;
```

```

CREATE TABLE order_summary AS
SELECT
    o.order_id,
    o.customer_id,
    c.customer_unique_id,
    c.customer_city,
    c.customer_state,
    o.order_status,
    o.order_purchase_timestamp,
    ROUND(SUM(oi.price)::numeric, 2) AS product_revenue,
    ROUND(SUM(oi.freight_value)::numeric, 2) AS freight_revenue,
    ROUND(SUM(oi.price + oi.freight_value)::numeric, 2) AS total_order_value,
    COUNT(*) AS item_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY
    o.order_id, o.customer_id, c.customer_unique_id,
    c.customer_city, c.customer_state, o.order_status,
    o.order_purchase_timestamp;

SELECT * FROM order_summary LIMIT 10;

```

Revenue queries

```

SELECT
    ROUND(SUM(product_revenue)::numeric, 2) AS total_product_revenue
FROM order_summary;

WITH monthly AS (
    SELECT
        DATE_TRUNC('month', order_order_purchase_timestamp)::date AS month,
        ROUND(SUM(product_revenue)::numeric, 2) AS monthly_revenue,
        COUNT(DISTINCT order_id) AS monthly_orders
    FROM order_summary
    GROUP BY 1
),
with_growth AS (
    SELECT
        month,
        monthly_revenue,
        monthly_orders,
        LAG(monthly_revenue) OVER (ORDER BY month) AS prior_month_revenue
    FROM monthly
)
SELECT
    month,
    monthly_revenue,
    monthly_orders,
    ROUND(
        100.0 * (monthly_revenue - prior_month_revenue)
        / NULLIF(prior_month_revenue, 0), 2
    ) AS mom_revenue_growth_pct
FROM with_growth
ORDER BY month;

-- Overall average order value

```

```

SELECT
  ROUND(AVG(product_revenue)::numeric, 2) AS overall_avg_order_value
FROM order_summary;

-- Monthly average order value
SELECT
  DATE_TRUNC('month', order_order_purchase_timestamp)::date AS month,
  ROUND(AVG(product_revenue)::numeric, 2) AS monthly_avg_order_value
FROM order_summary
GROUP BY 1
ORDER BY 1;

```

Customer queries

```

WITH customer_orders AS (
  SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id) AS order_count
  FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  GROUP BY c.customer_unique_id
)
SELECT
  COUNT(*) AS unique_customers,
  COUNT(*) FILTER (WHERE order_count > 1) AS repeat_customers,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE order_count > 1) / COUNT(*),
    2
  ) AS repeat_purchase_rate_pct
FROM customer_orders;

SELECT
  customer_state,
  ROUND(SUM(product_revenue)::numeric, 2) AS product_revenue,
  COUNT(DISTINCT order_id) AS orders,
  ROUND(AVG(product_revenue)::numeric, 2) AS avg_order_value
FROM order_summary
GROUP BY customer_state
ORDER BY product_revenue DESC;

```

Product and seller queries

```

WITH category_sales AS (
  SELECT
    COALESCE(p.product_category_name, 'unknown') AS category,
    SUM(oi.price) AS category_revenue
  FROM order_items oi
  LEFT JOIN products p ON oi.product_id = p.product_id
  GROUP BY 1
),
total AS (
  SELECT SUM(category_revenue) AS total_revenue
  FROM category_sales
)
SELECT
  category,

```

```

    ROUND(category_revenue::numeric, 2) AS category_revenue,
    ROUND(100.0 * category_revenue / total_revenue, 2) AS revenue_share_pct
FROM category_sales
CROSS JOIN total
ORDER BY category_revenue DESC
LIMIT 10;

```

```

SELECT
    COALESCE(p.product_category_name, 'unknown') AS category,
    COUNT(*) AS item_count,
    COUNT(DISTINCT oi.order_id) AS order_count
FROM order_items oi
LEFT JOIN products p ON oi.product_id = p.product_id
GROUP BY 1
ORDER BY item_count DESC
LIMIT 10;

```

```

SELECT
    seller_id,
    ROUND(SUM(price)::numeric, 2) AS seller_revenue,
    COUNT(DISTINCT order_id) AS order_count,
    COUNT(*) AS items_sold
FROM order_items
GROUP BY seller_id
ORDER BY seller_revenue DESC
LIMIT 10;

```

```

WITH seller_sales AS (
    SELECT
        seller_id,
        SUM(price) AS revenue
    FROM order_items
    GROUP BY seller_id
),
ranked AS (
    SELECT
        seller_id,
        revenue,
        ROW_NUMBER() OVER (ORDER BY revenue DESC) AS seller_rank
    FROM seller_sales
),
totals AS (
    SELECT SUM(revenue) AS total_revenue FROM seller_sales
)
SELECT
    ROUND(100.0 * SUM(revenue) FILTER (WHERE seller_rank <= 10)
        / MAX(total_revenue), 2) AS top_10_share_pct,
    ROUND(100.0 * SUM(revenue) FILTER (WHERE seller_rank <= 25)
        / MAX(total_revenue), 2) AS top_25_share_pct
FROM ranked
CROSS JOIN totals;

```

Delivery and review queries

```

SELECT
    ROUND(AVG(EXTRACT(EPOCH FROM
        (order_delivered_customer_date - order_purchase_timestamp)
    ) / 86400.0)::numeric, 2) AS avg_delivery_days,

```

```

ROUND(
  100.0 * COUNT(*) FILTER (
    WHERE order_delivered_customer_date > order_estimated_delivery_date
  ) / COUNT(*),
  2
) AS late_delivery_rate_pct
FROM orders
WHERE order_delivered_customer_date IS NOT NULL
AND order_purchase_timestamp IS NOT NULL
AND order_estimated_delivery_date IS NOT NULL;

SELECT
CASE
  WHEN o.order_delivered_customer_date <= o.order_estimated_delivery_date
    THEN 'On time'
  ELSE 'Late'
END AS delivery_status,
COUNT(*) AS review_count,
ROUND(AVG(r.review_score)::numeric, 2) AS avg_review_score
FROM orders o
JOIN reviews r ON o.order_id = r.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY 1
ORDER BY 1;

```

Payment queries

```

WITH payment_summary AS (
  SELECT
    payment_type,
    COUNT(*) AS payment_records,
    SUM(payment_value) AS total_payment_value,
    AVG(payment_value) AS avg_payment_value
  FROM payments
  GROUP BY payment_type
),
totals AS (
  SELECT SUM(payment_records) AS total_records FROM payment_summary
)
SELECT
  payment_type,
  payment_records,
  ROUND(100.0 * payment_records / total_records, 2) AS payment_method_share_pct,
  ROUND(total_payment_value::numeric, 2) AS total_payment_value,
  ROUND(avg_payment_value::numeric, 2) AS avg_payment_value
FROM payment_summary
CROSS JOIN totals
ORDER BY payment_records DESC;

-- Overall installment metrics
SELECT
  ROUND(AVG(payment_installments)::numeric, 2) AS avg_installments,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE payment_installments > 1) / COUNT(*),
    2
  ) AS share_using_installments_pct
FROM payments

```

```
WHERE payment_installments IS NOT NULL;

-- Average payment value by installment count
SELECT
  payment_installments,
  COUNT(*) AS payment_records,
  ROUND(AVG(payment_value)::numeric, 2) AS avg_payment_value
FROM payments
WHERE payment_installments IS NOT NULL
GROUP BY payment_installments
ORDER BY payment_installments;
```

Source Note

Olist, “Brazilian E-Commerce Public Dataset by Olist,” Kaggle. The dataset description identifies the release as anonymized commercial marketplace data with approximately 100,000 orders from 2016-2018.